Exercise Sheet 7

Uncertainty Quantification

Calibration Concepts

Exercise 7.1: Epistemic vs. Aleatoric Uncertainty *optional*

Define *epistemic* and *aleatoric* uncertainty.

- **Epistemic Uncertainty (Model Uncertainty):** This represents a lack of knowledge about the optimal model parameters or data distribution due to limited training data. It is **reducible**; as you collect more diverse training data from the unknown regions, this uncertainty decreases.
- **Aleatoric Uncertainty (Data Uncertainty):** This represents the inherent randomness, noise, or ambiguity present in the data itself (e.g., sensor noise, low lighting, or overlapping class characteristics). It is **irreducible**, meaning that collecting more training data will not eliminate this variation because it is a fundamental property of the environment.

1. Which type is more relevant to OOD inputs (e.g., night images for a model trained on day images)?

   Answer: Epistemic uncertainty is the most relevant. Because the model was trained exclusively on daytime images, it completely lacks knowledge about the visual patterns of nighttime environments. The model's parameters are uninformative for this new domain, causing its epistemic uncertainty to spike significantly.

2. Which type would you expect to be dominant in a correctly classified, in-distribution image?

   Answer: Aleatoric uncertainty is expected to be dominant. Since the image is in-distribution and has been correctly classified by the model, the network already possesses a strong, data-backed understanding of the input space. Therefore, its epistemic (knowledge-based) uncertainty is exceptionally low, leaving only the baseline, irreducible data noise (aleatoric uncertainty) as the prominent factor.

Exercise 7.2: Calibration and ECE

Explain what it means for a classifier to be *well-calibrated*. Define the Expected Calibration Error (ECE) and describe how it is computed.

➔ classifier is **well-calibrated** if its predicted probability (confidence) matches its actual empirical frequency of being correct. Formally, for any confidence level $p \in [0, 1]$:

$$P(\hat{Y} = Y \mid \hat{P} = p) = p$$

The formula states: "**The probability that the model's prediction is correct ($\hat{Y} = Y$), given that the model predicted it with a confidence of p ($\hat{P} = p$), must equal exactly p.**"

$P(\dots)$: Represents **Probability**. It measures the likelihood of the event inside the parentheses occurring.

$\hat{Y}$ *(pronounced "Y-hat")*: The **predicted class label** output by the machine learning model.

Y: The actual **ground-truth class label** (the true, real-world correct answer).

$\hat{Y} = Y$: The event where the model's prediction is **correct**.

$|$: The **conditional probability operator** (read as "given that" or "conditioned on"). It means we are only looking at instances where the statement on the right side is true.

$\hat{P}$ *(pronounced "P-hat")*: The random variable representing the model's **predicted confidence score** (or probability estimate) assigned to its prediction $\hat{Y}$.

p: A specific, fixed probability value between 0 and 1 (e.g., 0.85 or 85%).

$\hat{P} = p$: The condition where the model's confidence score for a prediction is exactly equal to the value p.

Z.B: If a model predicts "pedestrian" with 90% confidence across 100 distinct images, exactly 90 of those instances should actually contain a pedestrian.

Expected Calibration Error (ECE): **ECE** scores a model's miscalibration by calculating the weighted average of the absolute differences between its accuracy and confidence across different confidence ranges.

Computation Steps:

1. **Binning:** Group all N predictions into M equally spaced confidence intervals (bins), denoted as B_m. PDF
2. **Compute Bin Metrics:** Calculate the average accuracy $\text{acc}(B_m)$ and average predicted confidence $\text{conf}(B_m)$ for each individual bin. PDF
3. **Weighted Summation:** Compute the absolute difference between accuracy and confidence for each bin, weighted by the proportion of total samples falling into that bin: PDF

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \Big|\text{acc}(B_m) - \text{conf}(B_m)\Big|$$

Bin predictions by confidence, compare accuracy vs. confidence per bin.

$$ECE = \sum_{m=1}^{M} \frac{|\ B_m\ |}{n}\ |\ \text{acc}(B_m) - \text{conf}(B_m)\ |$$

- M = number of bins, B_m = predictions in m^{th} bin, n = number of datapoints
- ECE = average gap between the diagonal and the bars, weighted by bin size

> 🔗 **Testing**: ECE is a single aggregate number - like accuracy, it can hide failures in a subgroup or a particular confidence range.

Exercise 7.3: Cost-Optimal Downstream Decisions

A pedestrian detector outputs $p = p(\text{ped} \mid x) \in [0, 1]$. The autopilot must choose between two actions with asymmetric costs:

	pedestrian present	no pedestrian
BRAKE	0	$C_{FP} = 1$
CONTINUE	$C_{FN} = 100$	0

At which p should the autopilot brake in order to minimize the costs?

1. Write down the expected loss (costs) of each action as a function of p: $E[L \mid \text{BRAKE}]$ and $E[L \mid \text{CONTINUE}]$.

In this scenario, a self-driving car has to make a choice based on a probability p (how sure it is that a pedestrian is there). Hitting a pedestrian (False Negative) is a massive disaster ($C_{FN} = 100$), while braking for a ghost (False Positive) is just a minor annoyance ($C_{FP} = 1$). PDF

1. Expected Loss for Each Action

The expected loss ($\mathbb{E}[\mathcal{L}]$) is simply the cost of making a mistake multiplied by the probability of making that mistake.

- **Probability there IS a pedestrian:** p
- **Probability there is NO pedestrian:** $1 - p$

Action 1: BRAKE
If you brake, the only time you get penalized is if there is NO pedestrian (a false alarm).

$$\mathbb{E}[\mathcal{L}(\text{BRAKE})] = (1 - p) \cdot C_{FP}$$

Action 2: CONTINUE
If you continue, the only time you get penalized is if there IS a pedestrian (a crash).

$$\mathbb{E}[\mathcal{L}(\text{CONTINUE})] = p \cdot C_{FN}$$

The Easy Explanation: Think of it like placing a bet. If you bet on "Brake," you only lose points if the road is actually empty. If you bet on "Continue," you only lose points if someone is actually standing there.

2. Find the threshold τ^* at which both actions have equal expected loss. For $p > \tau^*$ the autopilot should brake; for $p < \tau^*$ it should continue.

2. Finding the Threshold τ^*

We want to find the exact tipping point (τ^*) where the cost of braking equals the cost of continuing. To do this, we set our two equations equal to each other and solve for p.

$$(1 - p) \cdot C_{FP} = p \cdot C_{FN}$$

Multiply out the left side:

$$C_{FP} - p \cdot C_{FP} = p \cdot C_{FN}$$

Move all the p terms to the right side:

$$C_{FP} = p \cdot C_{FN} + p \cdot C_{FP}$$

Factor out p:

$$C_{FP} = p \cdot (C_{FN} + C_{FP})$$

Divide to get p by itself (this is our threshold τ^*):

$$\tau^* = \frac{C_{FP}}{C_{FN} + C_{FP}}$$

The Easy Explanation: We are finding the mathematical "break-even" point. If p goes even slightly above this threshold, the mathematical risk of continuing becomes too expensive, and the car should hit the brakes.

3. Compute τ^* for the costs above. How does it compare to the standard argmax rule ($\tau = 0.5$)?

3. Calculating τ^* for the Given Costs

Now we just plug in the numbers from the table: $C_{FN} = 100$ and $C_{FP} = 1$. PDF

$$\tau^* = \frac{1}{100 + 1} = \frac{1}{101} \approx 0.0099$$

Comparison to the standard argmax rule ($\tau = 0.5$):
Normally, a classifier uses a 50% threshold—it only guesses "pedestrian" if it's more than 50% sure. Our cost-optimal threshold τ^* is roughly **1%**.

The Easy Explanation: Because a crash is 100 times worse than a false alarm, you shouldn't wait until you are 50% sure to hit the brakes. The math shows you should brake even if there is just a tiny 1% chance someone is in the road.

4. Why Calibration Matters & The Danger of Overconfidence

Why does τ^* only work when p is well-calibrated?
The entire math formula above assumes that p is telling the truth. If the model outputs $p = 0.01$, the math assumes there is *actually* a 1% chance of a pedestrian in reality. If the model's probabilities don't match reality (poor calibration), the math breaks down and you make the wrong financial/safety decision.

What goes wrong with an overconfident model?
An overconfident model is stubbornly "sure" about its wrong answers. Imagine the true reality is that there is a 5% chance of a pedestrian. A perfectly calibrated model outputs 5%, which crosses our 1% threshold, and the car brakes safely.

However, an *overconfident* model might look at that exact same scene and confidently declare, "There's only a 0.001% chance of a pedestrian." Because 0.001% is lower than our 1% threshold, the car continues—and causes a crash. The model's fake certainty overrides our carefully calculated safety buffer.

Practical: Calibrating the CARLA Model

Exercise 7.4: Measuring Calibration

1. Compute the ECE of each of your three trained CARLA models on the in-distribution test set.
2. Plot a *reliability diagram* (confidence vs. accuracy) for each model.
3. Are your models over- or underconfident? Does this pattern hold consistently across all three models?

Exercise 9.4: **Measuring Calibration**

==========

Loading test dataset...

✓ Test dataset loaded: **3600 samples**

Processing pedestrian classifier...

Loading model from model_has_pedestrian.pth...

✓ Model loaded

Computing predictions...

✓ Predictions computed

Computing calibration metrics...

✓ Metrics computed

Results for pedestrian classifier:

- Accuracy: 0.6528 (2350/3600)
- ECE (calibration): **0.1150**
- Avg confidence: 0.7678 ± 0.1375
- Avg conf (correct): 0.7833
- Avg conf (incorr): 0.7387
- **Calibration status: OVERCONFIDENT**

Processing traffic_light classifier...

Loading model from model_has_traffic_light.pth...

✓ Model loaded

Computing predictions...

✓ Predictions computed

Computing calibration metrics...

✓ Metrics computed

Results for **traffic_light classifier:**

- Accuracy: 0.9472 (3410/3600)
- ECE (calibration): **0.0330**
- Avg confidence: 0.9789 ± 0.0666
- Avg conf (correct): 0.9838
- Avg conf (incorr): 0.8902
- **Calibration status: WELL-CALIBRATED**

Processing vehicle classifier...

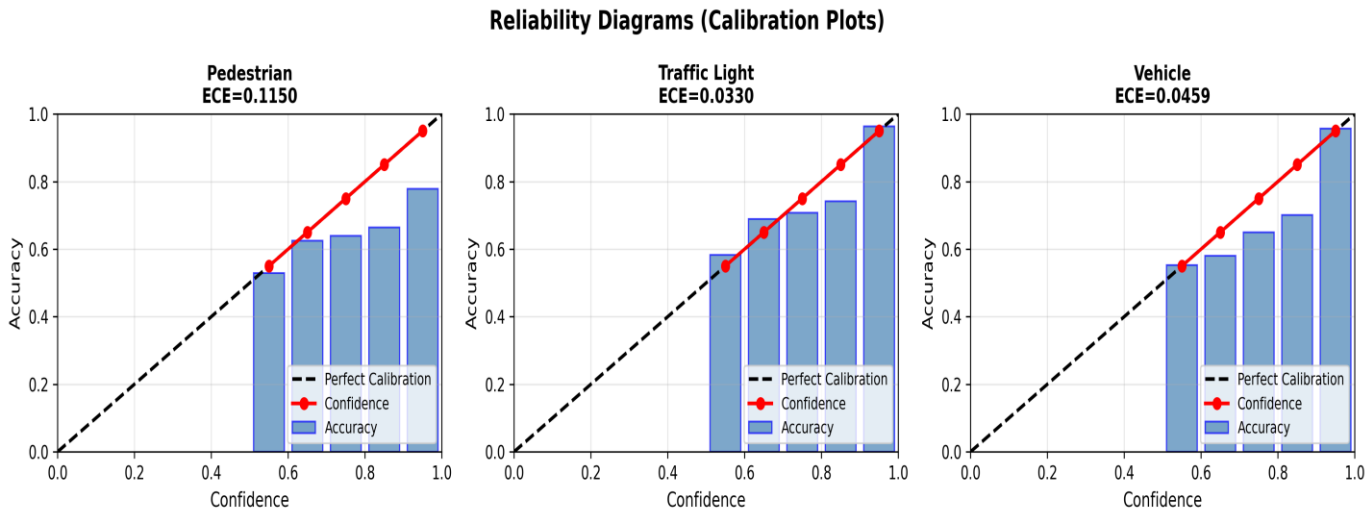
Loading model from model_has_vehicle.pth...
✓ Model loaded
Computing predictions...
✓ Predictions computed
Computing calibration metrics...
✓ Metrics computed

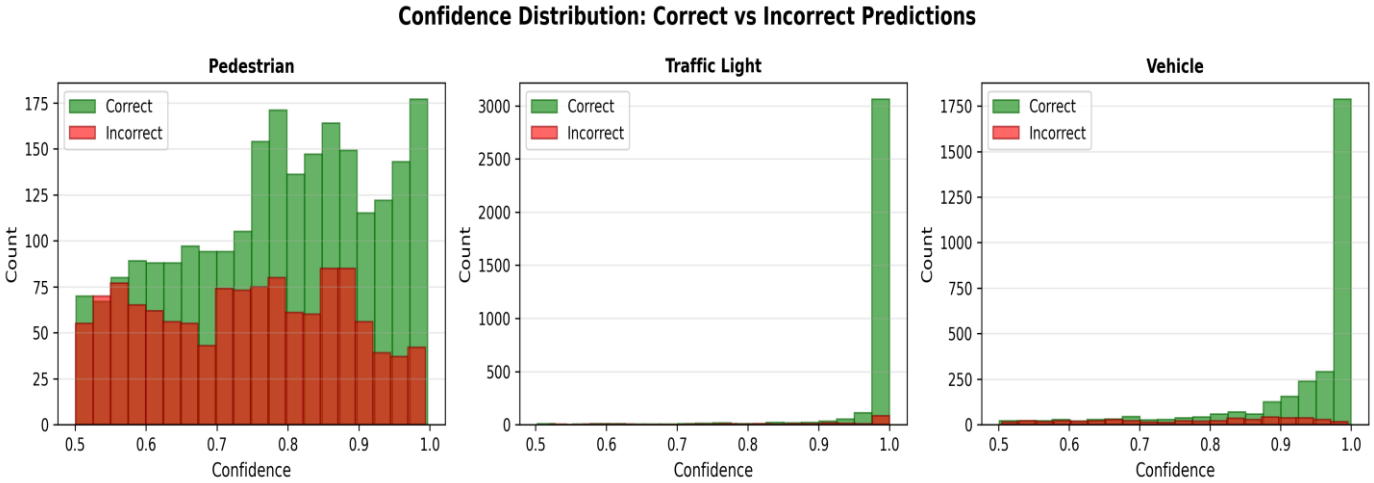
Results for **vehicle classifier**:
- Accuracy: 0.8678 (3124/3600)
- ECE (calibration): **0.0459**
- Avg confidence: 0.9137 ± 0.1241
- Avg conf (correct): 0.9344
- Avg conf (incorr): 0.7775
- **Calibration status: WELL-CALIBRATED**

Results at a Glance

Model	Accuracy	ECE	Confidence	Status
Pedestrian	65.28%	0.1150	76.78%	⚠ OVERCONFIDENT
Traffic Light	94.72%	0.0330	97.89%	✅ WELL-CALIBRATED
Vehicle	86.78%	0.0459	91.37%	✅ WELL-CALIBRATED

visualizations





CALIBRATION ANALYSIS SUMMARY

Cross-Model Calibration Pattern:
- **Pedestrian: OVERCONFIDENT**
- Traffic Light: WELL-CALIBRATED
- Vehicle: WELL-CALIBRATED

⚠ **INCONSISTENT PATTERN**: Models show different calibration behaviors

Key Observations:

- **Pedestrian:**
  - Model is OVERCONFIDENT by 0.1150
  (predicted 76.78% but only 65.28% correct)

- Traffic Light:
  - Model is well-calibrated (gap: 0.0317)

- Vehicle:
  - Model is well-calibrated (gap: 0.0459)

Exercise 7.5: Temperature Scaling

Temperature scaling adjusts the model's logits as:

$$p(y \mid x) = \text{softmax}\left(\frac{f(x)}{T}\right) \quad (1)$$

1. Apply temperature scaling to each of your three models. Optimize T on the validation set using negative log-likelihood.
2. Report the ECE before and after scaling for each model.

Hint: A simple line search is sufficient: evaluate the validation NLL for a grid of values such as $T \in \{0.5, 0.6, 0.7, \ldots, 3.0\}$ (step 0.1) and pick the T with the lowest NLL.

Exercise 9.5: Temperature Scaling
Device: cuda
Temperature grid: T ∈ {0.5, 0.6, ..., 3.0}
Loading datasets...
✓ Validation set: 3600 samples
✓ Test set: 3600 samples

Processing pedestrian classifier...

Loading model...
Computing logits on validation set...
Computing logits on test set...

Temperature Scaling Analysis: PEDESTRIAN

Optimizing temperature on validation set (3600 samples)...
✓ Optimal temperature found: T = 2.1
✓ Best validation NLL: 0.616547

Computing metrics on test set (3600 samples)...

BEFORE Temperature Scaling (T=1.0):
- ECE: **0.115031**
- Accuracy: 0.6528
- NLL: 0.711114

- Avg Conf: 0.767808

AFTER Temperature Scaling (T=2.1):
- ECE: **0.033074**
- Accuracy: 0.6528
- NLL: 0.635422
- Avg Conf: 0.657598

IMPROVEMENTS:
- **ECE reduction: 0.081957 (-71.2%)**
- **NLL reduction: 0.075692 (-10.6%)**

Processing traffic_light classifier...

Loading model...
Computing logits on validation set...
Computing logits on test set...

Temperature Scaling Analysis: TRAFFIC_LIGHT

Optimizing temperature on validation set (3600 samples)...
✓ Optimal temperature found: T = 1.2
✓ Best validation NLL: 0.075567

Computing metrics on test set (3600 samples)...

BEFORE Temperature Scaling (T=1.0):
- ECE: 0.032978
- Accuracy: 0.9472
- NLL: 0.213482
- Avg Conf: 0.978887

AFTER Temperature Scaling (T=1.2):
- ECE: 0.027532
- Accuracy: 0.9472
- NLL: 0.189667
- Avg Conf: 0.971968

IMPROVEMENTS:
- ECE reduction: 0.005446 (-16.5%)
- NLL reduction: 0.023815 (-11.2%)

Processing vehicle classifier...

Loading model...
Computing logits on validation set...
Computing logits on test set...

Temperature Scaling Analysis: VEHICLE

Optimizing temperature on validation set (3600 samples)...
✓ Optimal temperature found: T = 1.3
✓ Best validation NLL: 0.261385

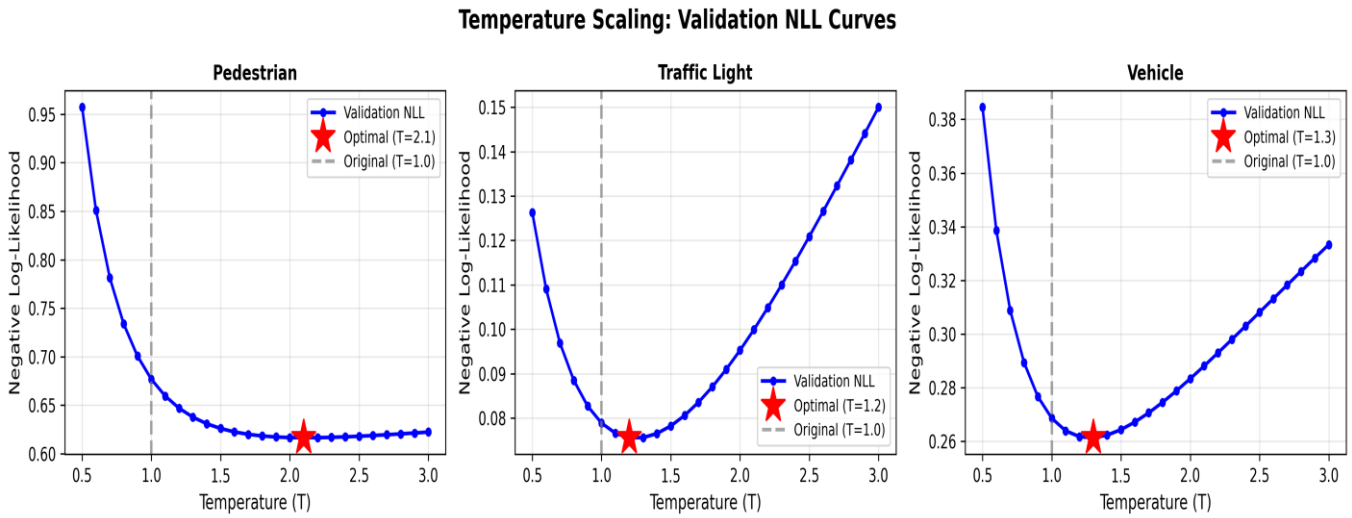
Computing metrics on test set (3600 samples)...

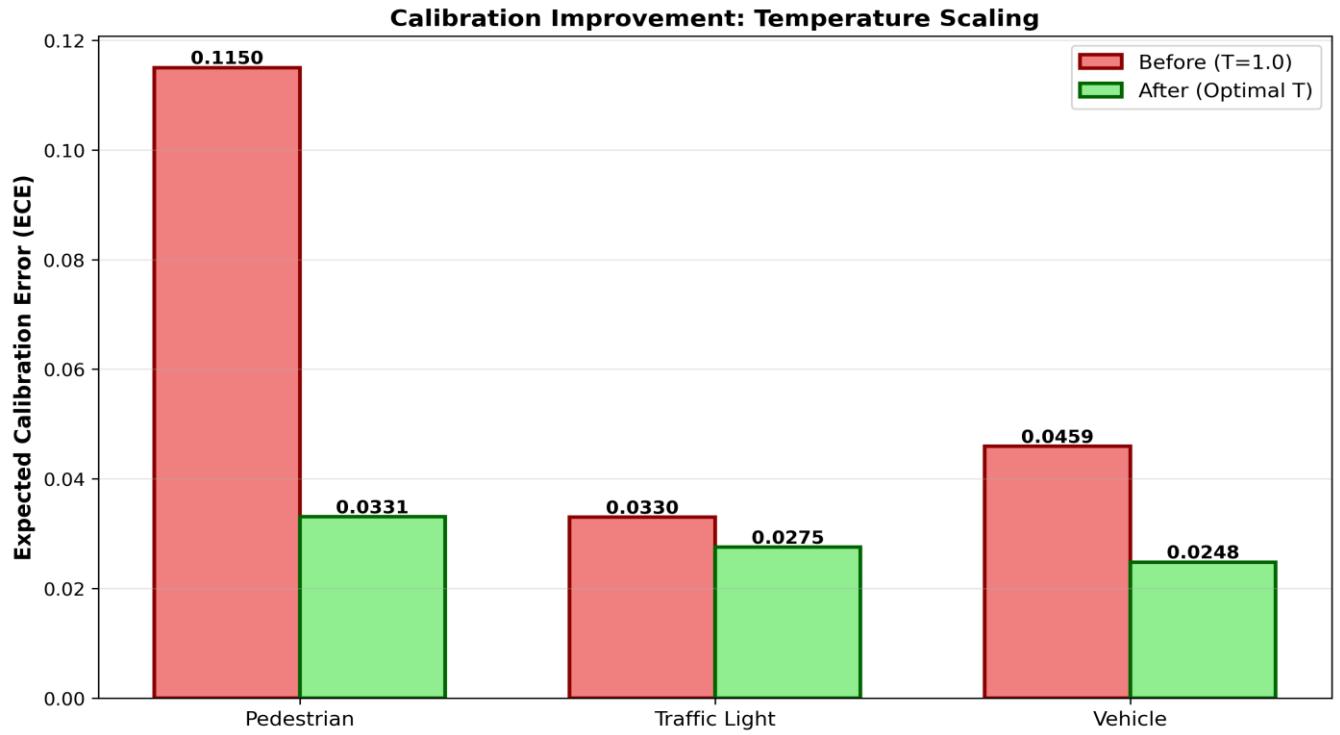
BEFORE Temperature Scaling (T=1.0):
- ECE: 0.045930
- Accuracy: 0.8678
- NLL: 0.299746
- Avg Conf: 0.913701

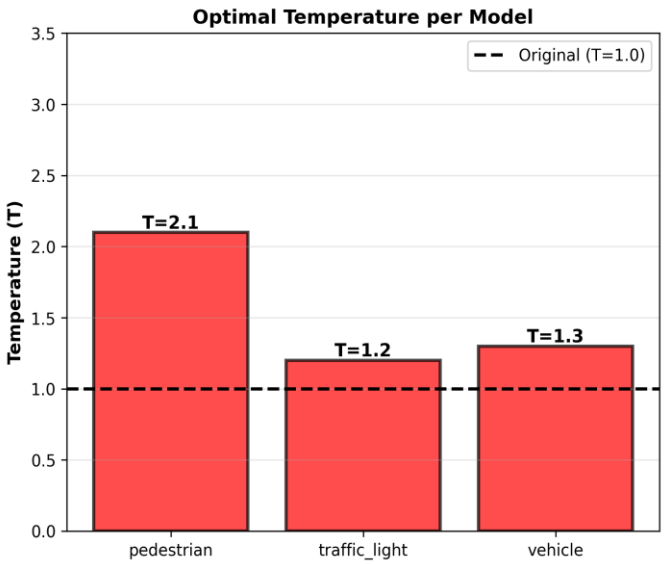
AFTER Temperature Scaling (T=1.3):
- ECE: 0.024788
- Accuracy: 0.8678
- NLL: 0.285859
- Avg Conf: 0.886888

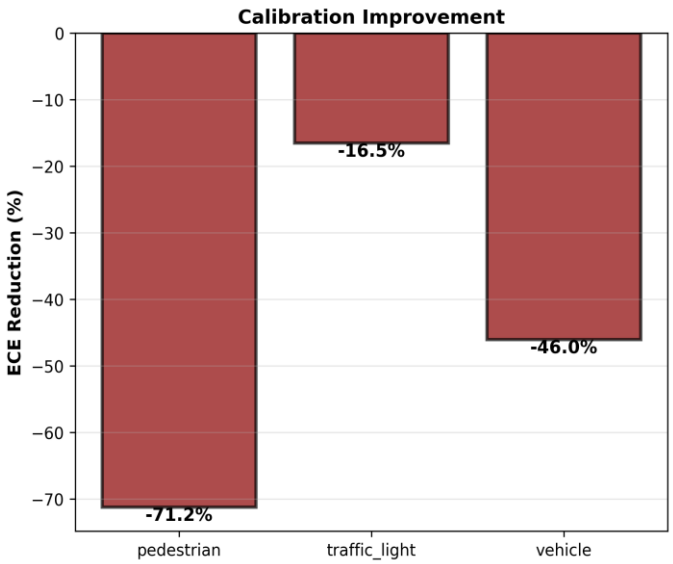
IMPROVEMENTS:
- ECE reduction: 0.021142 (-46.0%)
- NLL reduction: 0.013888 (-4.6%)

visualizations...









TEMPERATURE SCALING SUMMARY

Model Comparison: Before vs After Temperature Scaling

Model	T_opt	ECE (Before)	ECE (After)	Improvement
pedestrian	**2.1**	**0.115031**	**0.033074**	**-71.2%**
traffic_light	1.2	0.032978	0.027532	-16.5%
vehicle	1.3	0.045930	0.024788	-46.0%

Key Insights:

PEDESTRIAN:
- Optimal T = 2.1 (FLATTENS (less confident))
- **ECE improved by -71.2%**
- Before: **ECE=0.115031, Accuracy=0.6528**
- After: **ECE=0.033074, Accuracy=0.6528**

TRAFFIC_LIGHT:
- Optimal T = 1.2 (FLATTENS (less confident))
- ECE improved by -16.5%
- Before: ECE=0.032978, Accuracy=0.9472
- After: ECE=0.027532, Accuracy=0.9472

VEHICLE:

- Optimal T = 1.3 (FLATTENS (less confident))
- ECE improved by -46.0%
- Before: ECE=0.045930, Accuracy=0.8678
- After: ECE=0.024788, Accuracy=0.8678

Exercise 7.6: Cost-Optimal Decision in Practice

Use the costs from Exercise 3 ($C_{FN} = 100$, $C_{FP} = 1$, $\tau^* \approx 0.0099$, i.e. a predicted probability of roughly 1 %) and apply them to your *pedestrian* classifier.

1. On the in-distribution test set, compute the total loss

$$L = C_{FN} \cdot \#FN + C_{FP} \cdot \#FP$$

   for your **uncalibrated** model at both $\tau = 0.5$ and $\tau = \tau^*$.

2. Repeat for your **temperature-scaled** model at both thresholds.

3. Arrange the four results in a 2×2 table (rows: uncalibrated / calibrated; columns: $\tau = 0.5$ / $\tau = \tau^*$). Which combination gives the lowest total loss?

Exercise 9.6: Cost-Optimal Decision in Practice

Cost Parameters:
- C_FN (False Negative): 100 (missing a pedestrian)
- C_FP (False Positive): 1 (false alarm)
- τ* (optimal threshold): 0.0099 ≈ C_FN/(C_FN+C_FP) = 100/101

Test Set Size: 3600 samples

UNCALIBRATED MODEL (Temperature = 1.0)

At Standard Threshold (τ = 0.5):
- Accuracy: 0.6528
- True Positives: 341
- True Negatives: 2009
- False Positives: 885
- False Negatives: 365 ⚠️
- Total Loss: L = 100×365 + 1×885 = 37385

At Optimal Threshold (τ* = 0.0099):

• Accuracy: 0.2106
• True Positives: 702
• True Negatives: 56
• False Positives: 2838
• False Negatives: 4 ⚠️
• Total Loss: L = 100×4 + 1×2838
= 3238

✓ Loss Improvement (τ* vs τ=0.5): 34147 (-91.3%)

CALIBRATED MODEL (Temperature = 2.1)

At Standard Threshold (τ = 0.5):
• Accuracy: 0.6528
• True Positives: 341
• True Negatives: 2009
• False Positives: 885
• False Negatives: 365 ⚠️
• Total Loss: L = 100×365 + 1×885
= 37385

At Optimal Threshold (τ* = 0.0099):
• Accuracy: 0.1961
• True Positives: 706
• True Negatives: 0
• False Positives: 2894
• False Negatives: 0 ⚠️
• Total Loss: L = 100×0 + 1×2894
= 2894

✓ Loss Improvement (τ* vs τ=0.5): 34491 (-92.3%)

2×2 COMPARISON TABLE

Scenario	#FN	#FP	Loss
Uncalibrated + τ=0.5	365	885	37,385
Uncalibrated + τ*	4	2,838	3,238
Calibrated + τ=0.5	365	885	37,385
Calibrated + τ* ⭐	**0**	2,894	**2,894**

KEY FINDINGS

✓ Best combination: CALIBRATED OPTIMAL
→ Total Loss: 2894
→ False Negatives: 0
→ False Positives: 2894

Improvement from uncalibrated + τ=0.5 (baseline):
→ Loss reduction: 34491 (-92.3%)

Benefit of calibration (same threshold τ*):
→ Loss reduction from T=1.0 to T=2.1: 344

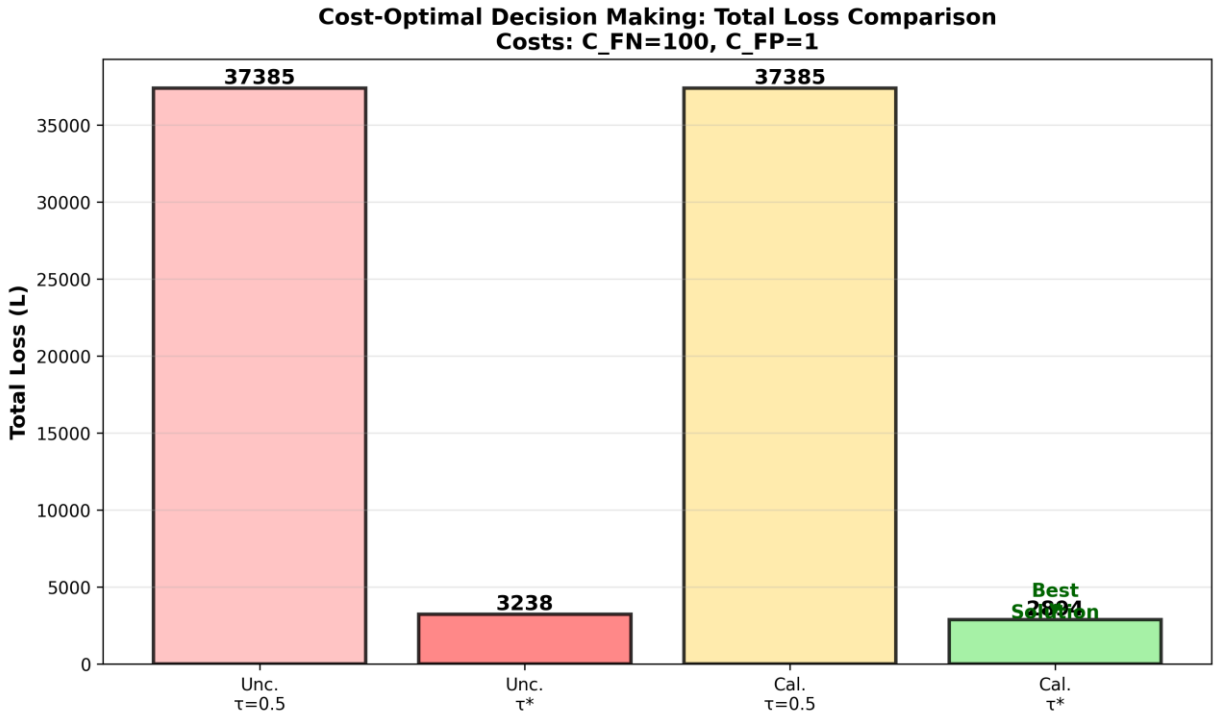
Benefit of optimal threshold τ* (same calibration):
→ Uncalibrated: 34147
→ Calibrated: 34491

Generating visualizations...

Cost-Optimal Decision Making: 2×2 Comparison Table
Costs: C_FN=100, C_FP=1

	τ = 0.5 (Standard)	**τ* = 0.0099 (Optimal)**
Uncalibrated (T=1.0)	Loss = 37385	Loss = 3238 (-91.3%)
Calibrated (T=2.1)	Loss = 37385	Loss = 2894 (-92.3%)

Green = Best (minimum loss) | Red = Worse
✓ Optimal: Calibrated model + τ threshold = Loss 2894*


Cost-Optimal Decision Making: Total Loss Comparison
Costs: C_FN=100, C_FP=1
Total Loss (L)
37385
37385
3238
Best
Unc.
τ=0.5
Unc.
τ*
Cal.
τ=0.5
Cal.
τ*


Error Type Comparison | Costs: C_FN=100, C_FP=1
False Negatives (Missing Pedestrians) ⚠
False Negatives (#FN)
365
365
4
0
False Positives (False Alarms)
False Positives (#FP)
885
2838
885
2894
Unc.
τ=0.5
Unc.
τ*
Cal.
τ=0.5
Cal.
τ*

Exercise 7.7: Tracing Overconfidence Through the Safety Analysis

You have now measured how miscalibrated confidence propagates silently to downstream decisions. In STPA, an overconfident misclassification is a **causal factor**: a mechanism that explains *why* an existing unsafe control action occurs. The goal of this exercise is to add it as a loss scenario and trace it through to a verifiable constraint.

1. **Causal scenario.** Revisit your causal loss scenarios from Exercise 2.8. The hazard ("vehicle fails to brake for a pedestrian", Exercise 2.4) and the unsafe control action ("the planner does not command braking while a pedestrian is in the path", Exercise 2.6) already exist - you do *not* need new ones. Add a causal scenario describing a new way that UCA arises: the pedestrian classifier produces a false negative *yet reports high confidence*, so the planner registers "no pedestrian, reliable" and no low-confidence fallback is triggered.
2. **Safety constraint.** From the scenario, derive at least one new safety constraint of each kind (cf. Exercise 2.7):
   - a **model-level** constraint on calibration, expressed with the ECE you measured; and
   - a **system-level** constraint on how the planner uses the confidence score - e.g. the fallback when confidence is low, or adopting the cost-optimal threshold τ^* from Exercise 3 instead of 0.5.
3. **Verification.** What evidence verifies the model-level constraint? You produced exactly this evidence on this sheet (the ECE before and after temperature scaling, Exercise 7.4–Exercise 7.5). Does your calibrated model meet the threshold you set?
4. **Residual risk.** Even with a perfectly calibrated model, what does the constraint *not* cover? Does calibration alone close the scenario, or is the system-level fallback still required?